NFC tag writer + amiibo emulator

User guide

charge on the bolt side

OPENTAG





Print it, fold it, keep it in the box. Regenerate with
python3 tools/make_booklet.py

Meet your OpenTag

OpenTag is a pocket handheld for NFC tags. It emulates amiibo with their own unique IDs, writes NDEF notes onto real tags, and emulates those too - no phone needed at tap time.

The tour

Where	What it is
Front	OLED screen, four buttons (UP, DOWN, OK, BACK)
Back	NFC tap window ringed by the halo - tap consoles and phones flat against it
Left side	USB-C charge port under the engraved lightning bolt
Right side	Power slide switch (full off for storage)
Bottom-left corner	Lanyard / keyring loop (4 mm bore)

The engraved bolt marks the charging side. The charger's own indicator LEDs glow through the same port slot - so you can see charging with the case closed.

Power & charging

Charging

- Plug any USB-C cable into the side port marked with the engraved lightning bolt.
- The charge LEDs on the charger module peek through the port slot, and a tiny acrylic light pipe just below the port carries the glow to the outside face: glowing = charging, dark = full or unplugged.
- The optional printed desk stand leans the OpenTag back about ten degrees with the screen up and readable; the charge port ends up just behind the stand's tall back panel, so it still charges while docked with the cable dropped behind.
- Optional wireless charging: a Qi receiver coil inside the back charges from any Qi pad - line the engraved ring on the back up with the pad, and a small bolt icon appears at the top-right of the screen while charging.
- Charge fully before first use. A full battery lasts many hours of emulation; the screen sleeps when the device is off.
- Battery gauge: if your OpenTag has the optional voltage divider fitted, the header shows the cell percentage - grey normally, red at or below 15 percent. Below 3.35 V it warns once (Battery low); below 3.25 V for half a minute it powers off by itself to protect the cell. Charging suspends those checks.

Turning it off

- Soft off: hold OK on the menu. BACK cancels within the countdown.
- Wake: hold OK for about a second.
- Storage / travel: use the power switch on the right side - soft off still drains a tiny current, the switch does not.
- Auto-sleep: left alone for 3 minutes (no buttons, no syncing, no tag on the writer), it powers itself off the same way - BACK cancels during the countdown, hold OK to wake.

Soft off is a convenience, not a battery disconnect. For a multi-week shelf life, flip the side switch.

Buttons & the menu

Input	What it does
UP / DOWN	Move through lists and menu items; hold to auto-repeat, speeding up as you hold
OK (short)	Select, start emulation, confirm
OK (hold 1 s)	At the menu: power off. In the library: the advanced write-to-tag path
BACK (short)	Cancel, stop emulation, go back one screen
BACK (hold 1 s)	In the library: flag the entry for UID randomization (R badge). At the menu: toggle travel mode
UP / DOWN while emulating	Hot-swap to the next amiibo without stopping (uses all 8 emulator slots)
OK or BACK	Stops an active emulation (any button stops the NDEF-note emulation)

The menu

Item	What it's for
Amiibo library	Browse your amiibo, emulate one (opens first)
Backup tag	Save a physical tag's dump to the library
Inspect tag	Identify any tag: type, unique ID, its text
BLE sync	Connect the web page or Android app
Write NDEF note	Write your note to a physical NTAG215
Emulate NDEF note	Present the note as a virtual tag
Clear NDEF note	Erase the stored note
USB drive	Share the library read-only over a USB cable
Factory reset	Wipe library, keys and settings (hold OK to confirm)

Backup, travel & reset

Backup tag (reader mode)

- Pick Backup tag and tap any amiibo (or other NTAG215) flat against the halo on the back.
- The device reads and checks the tag, then saves its dump to the library under the tag's own unique-ID name, ready to emulate. New-generation V3 figures save as full images.
- Reading a tag you already saved says Already saved and jumps to it; a rewritten tag with the same unique ID saves as a -2 copy.
- With the optional piezo buzzer fitted, the device chirps softly when a backup is saved and when a write verifies - silence means no piezo, not a failure.

Inspect tag

- Pick Inspect tag and tap any tag against the halo on the back: the screen shows the tag's unique ID, its chip type and size (NTAG213/215/216 or MIFARE UL), and the first words of any text or address stored on it - or No NDEF data for a blank.
- Tap another tag to re-scan; it never writes anything.
- Use it to tell blank tags from written ones, or to check what an unknown sticker says before you commit anything to it.

Travel mode

- Hold BACK for a second at the menu to toggle it; the screen says Travel mode on or off.
- On: the device stops advertising over Bluetooth, so phones cannot discover it. Everything else keeps working; the BLE sync item reminds you it's on.
- The setting survives reboots and firmware updates.

Factory reset

- The last menu item wipes the library, key files and settings back to as-built (travel mode included).
- Confirm by holding OK for about a second and a quarter. Any short press cancels, and the screen gives up after 15 seconds.
- There is no undo - re-load your dumps from the sync page afterwards.

Somewhere with strict radio rules? Travel mode plus the side power switch covers it.

Your amiibo library

Loading amiibo

- Open the BLE sync page or the Android app and upload your own NTAG215 dump files (.bin) or a .zip of them.
- Files are matched by name against a bundled 932-game amiibo database, so the library shows proper game titles and art in the browser; the device shows the tidied-up name.

Emulating

- Pick an entry and press OK. The screen shows the tag name and its slot; the emulator presents the dump with its own unique ID.
- Tap the console or phone flat against the halo on the back.
- While emulating, UP/DOWN hot-swaps to the neighbouring amiibo without stopping - each swap moves to the next of the emulator's eight slots, so back-to-back console taps can use different figures.
- Press OK or BACK to stop.

Random-UID re-sign

- Hold BACK on an entry to flag it (an R badge appears next to the name). Hold BACK again to unflag.
- Every emulation or hot-swap of a flagged entry re-signs it with a fresh random unique ID, so a console treats it as a brand-new figure each time - handy for games that limit rewards per tag.
- Needs your key files loaded (the screen says Random needs keys otherwise). V3 entries are excluded: their layout is not re-signable.

The V3 badge

Entries marked V3 (such as Kirby Air Riders figures) are the new-generation tags. They emulate perfectly on this device but cannot be written onto standard blank NTAG215 tags - there is no matching physical chip. That is a hardware limitation of the tag format, not a fault in your OpenTag.

Advanced: write to a physical tag

OpenTag can re-sign a dump for a blank NTAG215's own unique ID and write it, TagMo-style. This is optional - emulation needs none of it.

- Once, upload your key files (unfixed-info.bin, locked-secret.bin) over BLE - they are user-supplied and never bundled.
- In the library, hold OK on an entry to arm the write, then present a blank NTAG215 to the back window.
- The device re-signs, writes, and reads everything back to verify.

Locking a tag makes it read-only forever. Locking always needs a second, deliberate long-press on the device - it can never happen remotely or by timeout.

NDEF notes

A note is a short text or https:// link the device keeps in memory and can stamp onto tags over and over.

Menu item	What happens
Write NDEF note	Writes the note to a physical NTAG215 (read-back verified)
Emulate NDEF note	Presents the note as a virtual, read-only tag - phones read it like any sticker
Clear NDEF note	Erases the stored note

Set or change the note from the BLE sync page before writing. The note survives firmware updates.

open source - see LICENSE

Sync & updates

Web sync

Open the sync page in a Chromium browser, pick your OpenTag from the Bluetooth prompt, and you can upload amiibo dumps, manage the note, and browse the on-device library with artwork - remotely emulate or delete entries.

Pairing

The first phone to connect runs a short numeric pairing: a random six-digit code appears on the OpenTag screen, your phone shows the same code, and confirming it bonds the two. A mismatched code is dropped immediately. To re-pair, delete the bond in your phone's Bluetooth settings first.

Android app

The companion app does the same over BLE: pick .bin dumps or a .zip of them, upload, browse, and trigger emulation.

USB drive (read-only)

- Pick USB drive and plug the DevKit's own USB-C port into any computer: the OpenTag appears as a small drive holding a read-only copy of your amiibo library plus a README.
- Drag bins off the device with no app at all. Loading new dumps still goes over BLE - the drive is a snapshot, rebuilt each time you dock.
- OK or BACK ejects the drive (the USB port goes back to the serial console).

Firmware updates

Plug the device into a computer over USB and run python tools/update_opentag.py - it downloads the latest release and flashes only the app partition. Your amiibo library, keys, settings and note survive every update.

Over the air: send your Wi-Fi name and password once from the sync page's Settings, then press Update - the OpenTag fetches the latest release over Wi-Fi, flashes itself and reboots, everything intact. BACK cancels mid-download.

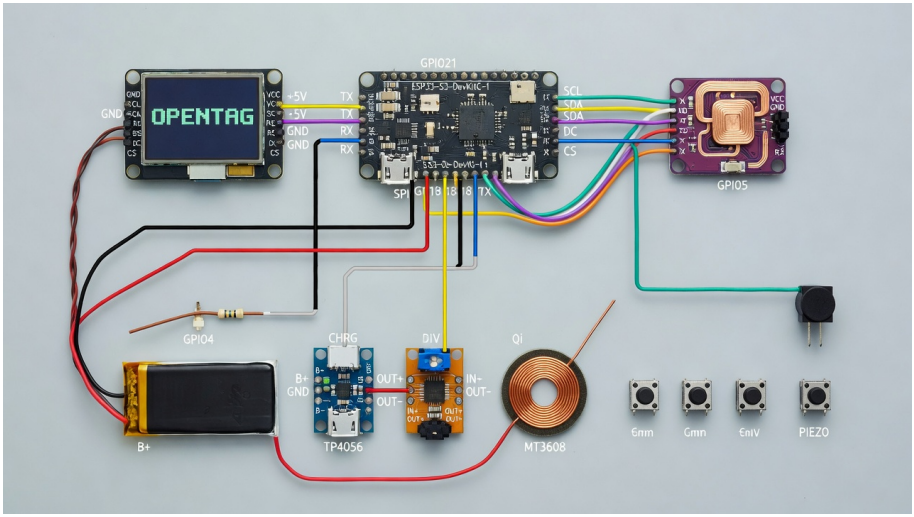
Troubleshooting

Symptom	Try this
Console reads nothing	Tap flat against the halo on the back; hold still a moment; check the emulation screen is showing
'Killer fw lacks V3' on screen	V3 emulation needs a current PN532Killer firmware - update the Killer board
'V3: emulation only'	By design: V3 amiibo cannot be written to physical tags
'PN532Killer absent'	Open the case and check the UART wiring (TX/RX swap is the usual cause)
BLE can't find the device	Exit any active emulation first; keep the phone within a few metres; toggle BLE off/on
Turned off by itself	That's auto-sleep: 3 minutes with no input. Hold OK about a second to wake where you left off
Won't wake	Hold OK a full second; if truly dead, charge it, then check the side power switch
Qi pad not charging	Centre the pad on the engraved ring on the back; the bolt icon on screen must appear - if not, reseat the pad or use another one
No battery percent shown	The gauge needs the optional voltage divider fitted inside - without it the header just shows the BLE state
Phone asks for a PIN	That's pairing: compare the six digits on the OpenTag screen with your phone and confirm. Mismatch? Cancel and retry
USB drive doesn't appear	Use the DevKit's own USB-C port (not the charge port), pick USB drive on the menu, and try another cable - some are charge-only
'OTA failed' on screen	The Wi-Fi credentials or the network were the problem - retry near the router, or use the wired updater instead

Charging but the LEDs look dark? Look at the port slot from an angle in dim light - the LEDs sit just inside, next to the connector.

Inside your OpenTag

Every module of the handheld and how they connect, on the open-source carrier board. The full wire-by-wire guide with pin tables lives in hardware/pcb/WIRING.md in the project repository - this is the map for identifying parts when you open the case.



Left to right: the OLED display on its 7-pin header, the ESP32-S3 controller at the heart, the PN532Killer NFC frontend on UART, and the power chain - battery, USB-C charger, boost module and the optional Qi receiver coil. Buttons and the optional piezo buzzer wire to the controller's general-purpose pins.

Care & safety

- The battery is a lithium-polymer cell: don't charge it unattended, don't puncture or squeeze the case, keep it off hot surfaces.
- If the case swells or the device gets unusually hot, stop using it and disconnect the battery (switch off, open the back).
- Wipe the case with a dry cloth; the tap window works best clean.
- Store with the side power switch off for long periods, battery about half full.

What's inside

An ESP32-S3 module, a PN532Killer NFC frontend, a 1.3-inch OLED, and a LiPo with a USB-C charge circuit - all on one open-source carrier board in a 3D-printed shell. Full schematics, CAD and firmware live in the project repository.

OpenTag is an open-source project. No Nintendo proprietary data is included: signing keys are user-supplied at runtime, and the amiibo database contains only public metadata matched by file name.